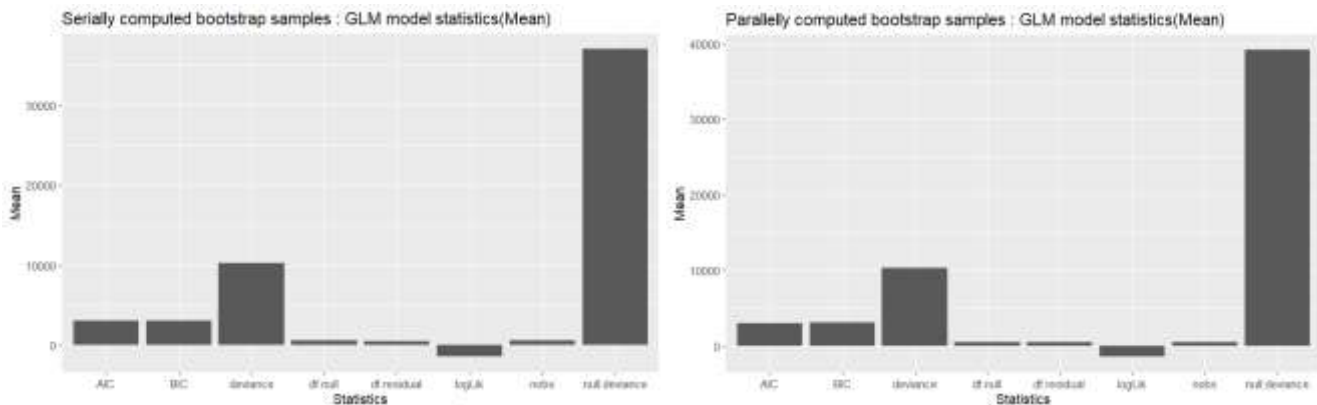


Week 02 - Parallelized Model Building

5.1. Are the distributions of the model fit statistics similar or different?

The distribution of the aggregated model statistics in the following plot seems identical; the reason is due to the usage of `set.seed` and features of the fitted model.



But if we want to inspect the numerical values of model statistics, we can see that even though we used the same seed, the aggregated statistics are slightly different. In serial execution, `set.seed()` ensures that the bootstrap samples are drawn in a predictable sequence. Whereas, in parallel execution, each processor or worker node may have their own set of independent random number streams, causing deviation in sampling. Therefore, even if `set.seed` is called, different workers may not receive the same seed. We can ensure reproducibility through the usage of the “doRNG” package, which ensures that each worker receives a unique but reproducible random seed.

Table 1: Comparison between serial and parallel computing with mean of GLM model statistics of 100 bootstrap sample

Statistics	Serial Computing	Parallel Computing
null.deviance	37056.002	39168.161
df.null	505	505
logLik	-1479.617	-1480.248
AIC	2989.235	2990.496
BIC	3052.633	3053.894
deviance	10269.669	10295.302
df.residual	492	492
nobs	506	506

5.2. Are the execution times for the serial and parallel approaches same or different?

Yes, it's different, and we can see that parallelization didn't work to our advantage this time. As each operation is simple, the overhead for each instance is taking more execution time instead. The following is the execution status of each step that required an iterative process, and we can see the difference in execution time for both operations.

Table 2 Difference in execution time for serial and parallel computing in seconds using Rprof

Steps	Serial computation (s)	Parallel computation (s)
Creating bootstrap samples	0.14	0.22
Fitting GLM model on samples	0.42	4.76
Retrieving model statistics of models	0.98	25.08

Also, I have used profiling through ‘microbenchmark’ and retrieved the following statistics, and it's evident that serial computing is far more efficient for this process than parallelization.

Table 3 Difference in execution time for serial and parallel computing in milliseconds using Microbenchmark

expr	min	lq	mean	median	uq	max	uq	max	neval	cld
<fctr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
serial_op	748.37	1883.2	3413.4	2225.4	3114.9	14465	3114.9	14465	10	a
parallel_op	33957	46828	99925	127286	135778	142405	135778	142405	10	b